CS 4501 Sign Language Captioning

Joseph Girard Joel Kishore Aaron Yuan
University of Virginia School of Engineering and Applied Science
85 Engineer's Way, Charlottesville, VA 22903
dfn4vu@virginia.edu, yvf2yd@virginia.edu, tta5rc@virginia.edu

Abstract

We propose a real-time sign-to-gloss captioning system that converts a live video stream of signing into an incremental sequence of gloss tokens. Unlike offline sign language recognition settings, live captioning must jointly optimize recognition quality, end-to-end latency, and output stability. Our approach uses efficient per-frame landmark extraction to obtain pose and hand keypoints, followed by a temporal encoder trained with Connectionist Temporal Classification (CTC) to predict gloss sequences without requiring frame-level alignment. CTC is well suited for unsegmented sequence labeling and enables streaming decoding through sliding-window inference and prefix-based commitment rules. We will evaluate the system on both isolated sign benchmarks and continuous signing corpora, and we will report standard recognition metrics together with streaming-focused measures, including latency, frames per second throughput, and caption churn. Our goal is to provide a practical and reproducible baseline for live gloss captioning and a principled characterization of the accuracy-latency-stability tradeoff in streaming sign recognition.

1. Introduction

This project aims to develop a sign language captioning system that translates an American Sign Language video input into live captions. The primary goal and idea of our project is to enable quick streaming transcription of sign language to make communication more accessible in live settings such as classrooms, meetings, and video calls.

To improve the system's speed, accuracy, and ability to eliminate background noise, we propose tracking the motion of the hands and body rather than analyzing each pixel in the video. A machine learning model will then process these sequences of hand and body positions over time to learn how signs change from frame to frame. Since we do not know exactly which frame corresponds to each word, we will use a method that allows the model to learn the correct sequence of outputs without needing exact labels for each frame.

The system will be using sliding windows from the incoming video frames, which will allow for incremental decoding and caption updates. This design helps the machine learning model generate live predictions while maintaining accuracy. To evaluate the performance of our system, we will test it in isolated and continuous sign language data sets and analyze its accuracy, speed, and consistency of predictions.

In addition to testing overall performance, we will focus on how well the system handles real-world challenges such as different signing styles and varying background noises.. We will also examine how consistent the captions remain as new frames are processed over time. The goal is to ensure that the system is not only accurate, but also reliable when used in live settings.

1.1. Literature Review

Sign language benchmarks for recognition and translation. Progress in sign language understanding has been accelerated by datasets that support both isolated sign recognition and continuous sequence modeling. Li *et al.* introduce WLASL, a large-scale word-level ASL benchmark with thousands of classes and substantial signer diversity, and they provide both appearance-based and pose-based baselines together with a temporal graph model for pose trajectories [5]. For continuous signing, Camgoz *et al.* introduce Neural Sign Language Translation and release RWTH-PHOENIX-Weather 2014T (PHOENIX14T), which includes gloss annotations aligned to continuous sign language videos and paired spoken-language translations [1]. These datasets motivate our project design: we will train and evaluate a streaming sign-to-gloss model on word-level data (as a controlled setting) and on continuous signing corpora (as the primary setting for gloss captioning).

Unaligned sequence learning with CTC. A central challenge in sign-to-gloss recognition is the lack of frame-level alignment between video frames and gloss tokens. Connectionist Temporal Classification (CTC) addresses this

issue by training sequence models to predict label sequences while marginalizing over monotonic alignments, eliminating the need for pre-segmented training data [4]. This alignment-free property makes CTC a natural fit for gloss transcription, where labels are typically available as token sequences rather than per-frame annotations. In our Plan A pipeline, we use a temporal encoder over landmark sequences and a CTC objective to learn gloss predictions directly from continuous input. This allows us to perform streaming inference using sliding windows and incremental decoding without requiring an explicit forced alignment stage.

Continuous sign modeling and transformer-based formulations. Beyond isolated sign classification, continuous sign language recognition requires modeling long-range temporal dependencies, coarticulation, and signer-specific variation. Camgoz *et al.* formulate sign language translation in a neural sequence-to-sequence framework and demonstrate that intermediate gloss supervision can substantially improve translation quality [1]. Building on this direction, Sign Language Transformers jointly address continuous sign language recognition and translation within a transformer architecture and bind the recognition and translation tasks through an explicit CTC loss, showing strong performance on PHOENIX14T [2]. These results support our choice of CTC-based gloss modeling as the backbone for live captioning. They also suggest a practical extension path: once a stable streaming gloss recognizer is in place, gloss outputs can be used as an intermediate representation for downstream translation, while maintaining the real-time constraints of the captioning interface.

Video representations and practical constraints for live captioning. Many vision systems for video understanding rely on heavy spatiotemporal backbones. Carreira and Zisserman show that strong 3D CNN representations, including I3D, benefit significantly from large-scale video pretraining and can transfer well to downstream action recognition settings [3]. However, live sign captioning introduces constraints that are often under-emphasized in offline benchmarks: end-to-end latency, throughput, and stability of partial outputs. To meet these constraints, our Plan A emphasizes efficient intermediate representations via pose and hand landmarks, paired with lightweight temporal encoders and streaming CTC decoding. Our evaluation will therefore complement standard recognition metrics (e.g., WER for gloss) with streaming-focused measurements, including latency, frames-per-second throughput, and caption stability under different window and decoding settings.

1.2. Datasets

Several possible datasets are suitable for this project's aim, with this discussion focusing on five alternatives. Each dataset contains depictions of American Sign Language (ASL), but they differ in how these depictions are stored, whether in photo or video format, as well as the specific ASL symbols that are depicted.

1. SignAlphaSet:
   This dataset was published on March 10, 2025, on Mendeley Data by Bharati Vidyapeeth Deemed University College of Engineering Pune and contains 26,000 images, with 1,000 images for each letter of the English alphabet in ASL. Across each letter's set of images, multiple different lighting conditions and finger placement variations are represented. The total size of the dataset is 422 MB, which is a slightly smaller-size dataset, which could limit the model's validation accuracy if an insufficient number of epochs are run. The dataset's limitation to just ASL letters could also be a potential issue when generalizing the model to a wide range of ASL classification problems. The dataset can be found at the following link: https://data.mendeley.com/datasets/8fmvr9m98w/1.
2. Images of American Sign Language (ASL) Alphabet Gestures:
   This dataset was published on December 16, 2024, on Mendeley Data by East West University, and holds 210,000 images that cover ASl letters, as well as the symbols for Delete and Space. In contrast to the prior dataset, this collection of gestures uses different backgrounds and three different individuals to capture real-world variability in sign language symbols. The total size of this dataset is 3.45 GB and it contains significantly more total images than the prior dataset, making it an even better choice for the complexity that will likely be required of our model. The dataset can be found at the following link: https://data.mendeley.com/datasets/48dg9vhmyk/2.
3. World Level American Sign Language:
   This dataset was published by a research group at the Australian National University to Kaggle and contains video representations of over 2,000 words in ASL. The dataset size is 5.4 GB, and given that it contains conversational words instead of only letters, it will likely be more generalizable to a greater range of captioning tasks. In addition, as each video contains different angles of each word, the dataset also captures a significant amount of real-world variation, suiting it well for our task. The dataset can be found at the following link: https://www.kaggle.com/datasets/risangbaskoro/wlasl-processed.

1.3. Plan of Activities

Week 1: Setup

- Finalize dataset selection and libraries
- Figure out communication and programming preferences

- Set up the initial project structure and core components

Weeks 2 and 3: Model Development

- Build a basic model to process hand and body positions
- Train and test the model on a small dataset
- Debug and find areas of improvement

Week 4: Model Improvement

- Tune the model according to the improvements found last week
- Begin testing the model with more complex datasets
- Debug and fix issues in the model

Weeks 5 and 6: Live Implementation

- Implement a sliding window to make live predictions
- Enable incremental caption updates from the video input
- Reduce noise effects and caption output errors

Week 7: Evaluate and Test

- Evaluate system performance on isolated and continuous datasets
- Analyze the accuracy, speed, and consistency of model predictions
- Test performance under various background noises
- Start the final report

Week 8: Finalization

- Prepare and finish the final report
- Generate a trial run for the presentation
- Final review of the code and minor tweaks

The workload will be divided evenly among group members, with adjustments depending on our alignment with the schedule.

2. Methods

2.1. System Overview

Our sign language captioning pipeline converts an input video into a sequence of textual predictions through three main stages: preprocessing, temporal modeling, and streaming inference. First, raw video frames are processed to extract structured hand and body motion information. Next, these processed features are passed into a temporal sequence model trained to predict gloss tokens over time. Finally, for live captioning, the model is applied in a sliding-window setting so that predictions can be updated incrementally as new frames arrive.

Rather than operating directly on raw RGB pixels, our system uses landmark-based motion features. This representation is more computationally efficient and better aligned with the structure of sign language, since the meaning of a sign depends strongly on the position and movement of the hands, arms, and upper body over time.

2.2. Landmark-Based Feature Extraction

To represent signing motion, we extract pose and hand landmarks from each frame of an input video. For each frame, the system records body and hand keypoints and concatenates them into a single feature vector. Stacking these frame-level vectors over time produces a sequence representation for the full input clip.

This landmark-based design reduces the dimensionality of the problem compared to pixel-based video modeling while preserving the information most relevant to sign production. In addition, using explicit hand and pose coordinates makes the system more robust to irrelevant visual details such as color, texture, and some background variation.

2.3. Feature Preprocessing

After landmark extraction, the resulting feature sequences are normalized before training. This step reduces variation in absolute position and scale across samples and makes the temporal patterns easier for the model to learn. In addition to normalized landmark coordinates, we incorporate frame-to-frame motion information by computing temporal differences between consecutive frames. As a result, each input sequence contains both spatial information about body and hand positions and temporal information about how those positions change over time.

The processed sequences are then saved and organized into training, validation, and test splits. This preprocessing pipeline allows the model to train on compact numerical representations rather than directly on raw video.

2.4. Temporal Modeling with BiLSTM-CTC

Our current model uses a bidirectional Long Short-Term Memory network (BiLSTM) combined with Connectionist Temporal Classification (CTC) loss. The BiLSTM is chosen because sign language is inherently sequential and depends on motion patterns that happen over time. By processing the sequence in both forward and backward directions, the model can capture richer and more enhanced temporal context before producing token predictions at each timestep.

CTC loss is used because the task does not give precise frame-level alignments between the input video and the required output sequence. Instead of requiring explicit annotation to determine which frame corresponds to which gloss token, CTC enables the model to learn alignments implicitly during training. This makes it ideal for sign language captioning, in which simply the general sequence label is typically provided.

2.5. Training Procedure

During training, feature sequences of different lengths are padded so that they can be processed efficiently in batches. The model is optimized using AdamW, and gradient clipping is applied to improve training stability. Validation performance is tracked throughout training, and model checkpoints are saved so that the best-performing version can

later be used for evaluation and streaming inference.

To convert model outputs into text predictions, we use greedy CTC decoding. At each timestep, the token with the highest probability is selected, repeated predictions are collapsed, and blank symbols are removed. Although simple, this decoding strategy provides a strong baseline and allows us to evaluate the effectiveness of the temporal model before exploring and implementing more advanced decoding methods.

2.6. Streaming Inference

To support live captioning, we implement a sliding-window inference strategy. Instead of waiting for an entire video to be processed before generating output, the system analyzes overlapping chunks of the feature sequence and updates predictions incrementally. This enables the model to produce partial captions in near real time.

Because streaming predictions can fluctuate as new frames arrive, we use a prefix commitment strategy to improve stability. In this approach, predicted tokens are only committed to the displayed caption when they remain consistent across repeated windows. This is intended to balance responsiveness with output stability, which is especially important in accessibility-focused real-time applications.

2.7. Implementation Details

The system was implemented as a landmark-based sign language recognition pipeline. Each input video was first converted into a sequence of frame-level pose and hand landmarks. These landmarks were then normalized to reduce differences in signer position, body scale, and camera placement. To better capture motion, we also included temporal-delta features, which represent how landmark positions change from one frame to the next.

After preprocessing, each video was represented as a compact numerical sequence rather than as raw RGB frames. This made training more efficient and helped the model focus on the hand, arm, and body movements most relevant to sign recognition. The processed sequences were split into training, validation, and test sets so that both models could be evaluated under the same conditions.

We implemented and compared two temporal encoders using the same input features and evaluation pipeline. The baseline model used a bidirectional LSTM encoder, while the improved model replaced the recurrent encoder with a Transformer encoder. Both models were trained using the same corrected preprocessing pipeline so that the main difference between the experiments was the temporal modeling architecture rather than the input representation.

During training, variable-length sequences were padded within each batch. We used the AdamW optimizer and gradient clipping to improve training stability. Model checkpoints were saved based on validation performance, and the best-performing checkpoint was used for final testing. At inference time, model outputs were decoded into predicted sign labels, and performance was measured using test loss, exact-match accuracy, token error rate, and top-5 accuracy.

This implementation design allowed us to isolate the effect of changing the temporal encoder. Since the preprocessing, dataset split, and evaluation metrics were kept fixed across both models, differences in final performance can be attributed mainly to the BiLSTM-to-Transformer change.

3. Experiments

3.1. Testbed and Experimental Goals

Our experiments were designed to answer one main question: after correcting the earlier training and inference issues in the pipeline, does replacing the BiLSTM-based temporal encoder with a Transformer-based temporal encoder lead to better isolated sign captioning performance? The comparison is important because both models operate on the same pose-based input representation, but they model temporal structure in different ways. The BiLSTM processes the sequence recurrently and summarizes information through hidden-state propagation, while the Transformer uses self-attention to compare all frames in a sequence directly.

All experiments were run on the same sign language captioning pipeline. The input to each model is a normalized sequence of landmark features extracted from video frames. The shared preprocessing and data pipeline were kept fixed across models so that differences in performance could be attributed to the temporal encoder rather than to differences in input format or evaluation procedure. This is especially important because earlier debugging in the project showed that poor results were not caused only by hyperparameters. Instead, the main failure mode came from a shared pipeline issue involving weak preprocessing, a poor fit between isolated-sign classification and pure CTC decoding, and configuration mismatches during inference. After those issues were corrected, the remaining comparison between the base model and the Transformer became meaningful.

The experiments in this section address the following questions:

1. Does the Transformer produce lower test loss than the BiLSTM baseline?
2. Does the Transformer improve exact-match caption accuracy?
3. Does the Transformer reduce token error rate?
4. Does the Transformer improve top-5 retrieval accuracy, which is especially relevant for ambiguous signs and large-vocabulary isolated recognition?

3.2. Compared Models

The first model is the base recurrent model, which uses a bidirectional LSTM encoder followed by prediction layers. Given an input sequence

$$X = [x_1, x_2, \ldots, x_T], \qquad x_t \in \mathbb{R}^d, \tag{1}$$

the BiLSTM computes forward and backward hidden states:

$$\overrightarrow{h_t} = \mathrm{LSTM}_f(x_t, \overrightarrow{h}_{t-1}), \qquad \overleftarrow{h_t} = \mathrm{LSTM}_b(x_t, \overleftarrow{h}_{t+1}), \tag{2}$$

and concatenates them:

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]. \tag{3}$$

A pooled sequence representation is then used for clip-level prediction, while the time-distributed outputs also support sequence modeling.

The second model replaces the recurrent encoder with a Transformer encoder. For each frame embedding, self-attention computes query, key, and value projections:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V. \tag{4}$$

The attention output is

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{5}$$

Multi-head self-attention allows the model to compare multiple parts of the signing sequence at once:

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H) W_O. \tag{6}$$

This is useful for sign language because the meaning of a sign often depends on nonlocal temporal relationships such as hand preparation, peak articulation, and recovery movement. Unlike the BiLSTM, the Transformer does not need to pass all temporal information through a chain of recurrent updates.

Both models were evaluated after the project-wide fixes introduced during development. These included stronger landmark normalization, local hand normalization, temporal-delta features, interpolation for missing hand detections, improved augmentation, weighted sampling, and a hybrid training design in which clip-level classification is the primary prediction path for isolated signs while CTC is retained as an auxiliary objective. These fixes were shared by both models. Therefore, the remaining difference measured here isolates the value of replacing the recurrent temporal encoder with a Transformer.

3.3. Evaluation Metrics

We report four metrics:

- **Test loss**: the final loss on the held-out test set.
- **Exact match**: the fraction of test samples for which the predicted caption exactly matches the target label.
- **Token error rate (TER)**: the fraction of output tokens that are incorrect, where lower is better.
- **Top-5 accuracy**: the fraction of samples for which the correct label appears among the five highest-scoring predictions.

The test loss over a test set of size N is defined as

$$\mathcal{L}_{\text{test}} = \frac{1}{N} \sum_{i=1}^{N} \ell(\hat{y}_i, y_i), \tag{7}$$

where $\ell(\hat{y}_i, y_i)$ is the CTC loss evaluated on the i-th test sample. For exact match over a test set of size N, the metric is

$$\mathrm{ExactMatch} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{\hat{y}_i = y_i\}. \tag{8}$$

Token error rate is reported as an empirical error statistic over predicted output tokens, represented as:

$$\mathrm{TER} = \frac{\sum_{i=1}^{N} D(\hat{y}_i, y_i)}{\sum_{i=1}^{N} |y_i|}, \tag{9}$$

where $D(\hat{y}_i, y_i)$ represents the minimum number of substitutions, insertions, and deletions required to change $\hat{y}_i$ into y_i, and $|y_i|$ is the length of the reference sequence. Top-5 accuracy is

$$\mathrm{Top5} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{y_i \in \mathcal{K}_5(\hat{p}_i)\}, \tag{10}$$

where $\mathcal{K}_5(\hat{p}_i)$ denotes the set of five most likely predicted classes for sample i.

3.4. Results

Table 1 shows the final test-set results for the base BiLSTM model and the Transformer model.

Model	Test Loss	Exact Match	TER	Top-5
Base BiLSTM	5.0575	0.0916	0.9084	0.3015
Transformer	4.3848	0.3359	0.6641	0.5711

Table 1. Final test-set comparison between the base BiLSTM model and the Transformer model. Lower is better for test loss and token error rate. Higher is better for exact match and top-5 accuracy.

The Transformer outperformed the base model on every reported metric. The test loss dropped from 5.0575 to 4.3848, an absolute reduction of 0.6727. This provides a relative improvement from the base BiLSTM model to the Transformer model of

$$\frac{5.0575 - 4.3848}{5.0575} = 0.1330 \tag{11}$$

or roughly a 13.3% reduction. Exact-match accuracy improved from 0.0916 to 0.3359. This is an absolute gain of 0.2443, which corresponds to a relative improvement of approximately

$$\frac{0.3359 - 0.0916}{0.0916} \approx 2.67, \tag{12}$$

or about 267% relative to the baseline exact-match rate.

Token error rate also improved substantially, decreasing from 0.9084 to 0.6641. The absolute reduction is 0.2443, and the relative error reduction is

$$\frac{0.9084 - 0.6641}{0.9084} \approx 0.269, \tag{13}$$

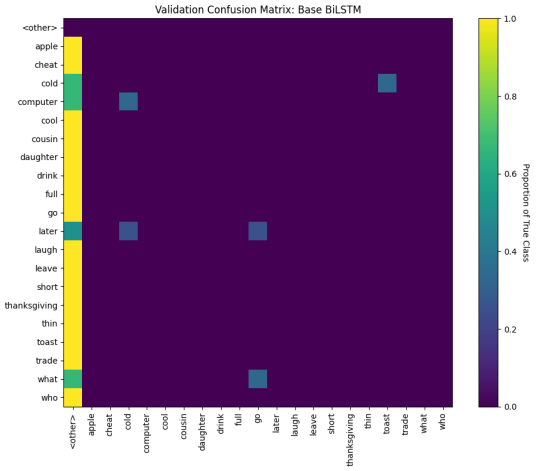
or about 26.9%. This result matters because earlier versions of the system had error rates near failure levels across both architectures. After the pipeline corrections, the Transformer was able to convert the improved features into substantially more reliable predictions than the recurrent baseline.

Top-5 accuracy rose from 0.3015 to 0.5711, an absolute gain of 0.2696. The relative improvement is

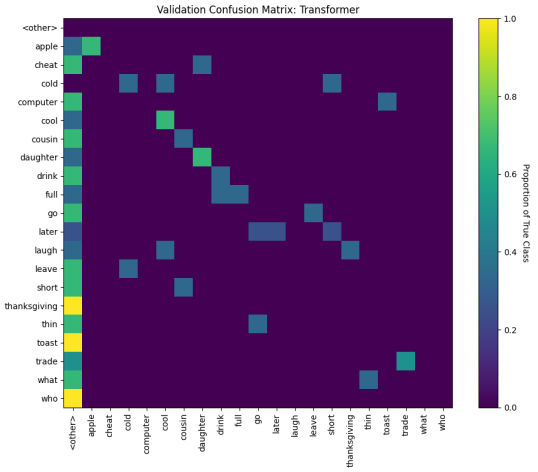
$$\frac{0.5711 - 0.3015}{0.3015} \approx 0.894, \tag{14}$$

or about 89.4%. This is a strong sign that the Transformer is not only improving the single best prediction, but also learning a more useful ranking of likely candidate signs. In a sign language captioning setting, this is important because some signs share similar hand shapes or partial motion patterns, and a good model should assign high probability to semantically close candidates even when the top-1 prediction is wrong.

A brief visualization of both models' validation accuracies as confusion matrices can be found below this section. For both models, validation accuracy across most signs is poor, but noticeable improvements can be observed in the Transformer model's figure, particularly along the main diagonal (top left to bottom right), which represents classifications where the true and predicted labels are the same. For the Transformer model, the proportion of times where the predicted labels (the x-axis) matched the true labels (the y-axis) increased compared to the base BiLSTM model. The base BiLSTM model has no correct classifications of the signs shown in the confusion matrix, classifying most signs as "other." A perfectly accurate model would have a confusion matrix with a main diagonal where each cell predicts the true sign 100% of the time (a diagonal with all yellow cells), and guesses any other sign 0% of the time (all other cells are purple). The Transformer model does have a wider variety of misclassifications compared to the base BiLSTM model, but its higher proportion of correct classifications illustrates a clear improvement over the base model.



(a) Validation Confusion Matrix for Base BiLSTM Model



(b) Validation Confusion Matrix for Transformer Model

Figure 1. Comparison of validation confusion matrices.

3.5. Analysis of the BiLSTM-to-Transformer Improvement

The results suggest that the main benefit of the Transformer is better temporal reasoning over the landmark sequence. In sign language data, informative cues can be distributed across the clip. A model may need to compare early hand preparation, mid-sequence articulation, and late recovery motion. The recurrent BiLSTM must compress earlier information into hidden states over time, while the Transformer can form direct pairwise comparisons between distant frames through self-attention.

This distinction becomes especially important after the shared pipeline fixes. Before those fixes, both models failed badly because the system as a whole was learning from poor features and from an objective formulation that did not align well with isolated-sign recognition. Once the shared is-

sues were corrected, the temporal encoder became the main source of difference. The Transformer then showed that it could make better use of the improved normalized hand and arm motion features than the BiLSTM could.

A second reason for the gain is that sign recognition often contains simultaneous or overlapping cues. Hand trajectory, relative hand position, arm movement, and local hand shape may all contribute to the class decision. Self-attention allows the model to weigh multiple informative frames jointly rather than relying on a compressed sequential memory. In practice, this makes the Transformer less sensitive to where the most important evidence appears in the clip.

The gap in top-5 accuracy is also revealing. The Transformer improved top-5 accuracy to 57.11%, while the BiLSTM reached only 30.15%. This suggests that even when the Transformer makes an incorrect top-1 decision, its internal representation is still organizing the label space more meaningfully. In other words, it is usually wrong in a more informed way. That behavior is consistent with a model that captures richer temporal structure.

3.6. Answering the Experimental Questions

The experiments provide clear answers to the questions posed at the beginning of this section.

First, the Transformer achieved lower test loss than the base model. This indicates a better overall fit to the held-out data. Second, the Transformer improved exact-match performance dramatically, moving from roughly 9.16% to 33.59%. Third, it reduced the token error rate by about 27%, from 0.9084 to 0.6641, indicating that the Transformer model improved the classification accuracy of very granular features. Fourth, it improved the top-5 classification accuracy by 89.4%, nearly doubling it compared to the base BiLSTM model. This indicates that our model is able to provide classification results that are at least contextually similar to the true sign, even when it is not able to provide an exact match. Taken together, these results show that the switch from a BiLSTM encoder to a Transformer encoder is not simply a cosmetic architecture change; rather, it materially improves classification performance once the rest of the pipeline is corrected.

The experiments also clarify an important lesson from the debugging process. The original near-failure behavior across models was not primarily caused by a bad learning rate, dropout choice, or optimizer setting. If poorly calibrated hyperparameters had been the primary problem, we would not expect both the base model and the Transformer to collapse in similar ways. Instead, the underlying issue was a shared modeling and preprocessing failure. The model's underperformance was driven by the encoder's poor compatibility to the complex task of sign-language classification, which involves complex sequences of hand positions and movements. The Transformer encoder uses self-attention, which improves its ability to learn different hand movements over a long sequence of motions, even if they appear in two different sections of the clip. Self-attention enables the model to learn how different signs relate to one another throughout a continuous stream of movements, which better fits the task of translating and captioning ASL signs as English words. After fixing those shared problems, the Transformer emerged as the stronger temporal model. This means the final system improvement came from two stages: first, correcting the broken shared pipeline, and second, replacing the BiLSTM with a Transformer to better exploit the repaired input representation.

3.7. Qualitative Error Analysis

In addition to the numerical metrics, the confusion matrices provide useful insight into how the two models behave. The base BiLSTM model showed weak class separation and often collapsed toward a small number of repeated predictions. This suggests that the recurrent baseline was not consistently learning distinct motion patterns for each sign. Instead, many different input videos were mapped to similar incorrect outputs, which helps explain the low exact-match accuracy and high token error rate. This also explains the poor performance of the base BiLSTM model in the confusion matrix, as numerous ASL signs, like "daughter," "computer," "thanksgiving," and "trade," all of which have very different grammatical meanings and roles, were incorrectly labeled as "other" by the base model.

The Transformer model produced a more meaningful distribution of predictions. Although it still made many mistakes, the confusion matrix showed more correct classifications along the main diagonal. This means that the Transformer was better able to match predicted labels with the true labels compared to the BiLSTM baseline. This qualitative pattern supports the quantitative results, where the Transformer improved exact-match accuracy, token error rate, and top-5 accuracy.

A likely reason for the remaining errors is that many signs share similar visual and temporal structure. Some signs may have similar hand shapes, similar arm trajectories, or only small differences in hand orientation. Since our system relies on pose and hand landmarks rather than full RGB information, it may lose subtle details such as finger shape, palm direction, or facial expression. These details can be important in sign language recognition, especially when two signs differ by only a small movement or hand configuration.

Another source of error is landmark quality. If the landmark detector fails to locate a hand in certain frames, produces noisy coordinates, or loses track of a fast motion, the temporal model receives an incomplete representation of the sign. Even though interpolation and normalization

help reduce this issue, particularly by reconstructing partially undetected hand positions, they cannot fully recover information that was missing from the original landmark extraction step. This means that some classification errors may come from the input representation rather than the temporal encoder alone.

The difference between top-1 and top-5 accuracy is also important. The Transformer achieved a much higher top-5 accuracy than the BiLSTM, which suggests that even when its top prediction was wrong, the correct label was often still ranked among its most likely outputs. This indicates that the Transformer learned a more useful internal representation of the sign classes. In other words, the model was often wrong in a more informed way, placing the correct answer near the top rather than assigning probability randomly across labels.

Overall, the qualitative results suggest that the Transformer improved both prediction accuracy and class organization. However, the remaining errors show that better temporal modeling alone is not enough to fully fit the model to the task. Future improvements will likely require stronger landmark extraction, more training data, better handling of subtle hand-shape differences, and possibly the inclusion of additional visual features beyond pose and hand coordinates.

3.8. Limitations and Future Experimental Directions

Although the Transformer clearly outperformed the base model, the final exact-match accuracy of 33.59% still leaves substantial room for improvement. An important limitation of this project is that model development was constrained by the practical limits of a university course setting. In particular, long end-to-end pipeline runtimes made experimentation expensive. Preprocessing, landmark extraction, training, and evaluation could take many hours on a standard non-enterprise device, which significantly limited the number of full runs that could be completed. As a result, it was not feasible to explore as many architecture variants, hyperparameter combinations, or repeated trials as would normally be desirable in a larger research setting.

A second limitation is compute. The project did not have access to large-scale training infrastructure or sustained high-end GPU resources. This affected both model design and experimental scope. For example, larger Transformer variants, longer training schedules, broader hyperparameter sweeps, and repeated seed-based evaluations would likely improve the reliability of the conclusions, but these were outside the realistic compute budget for the project. In practice, this meant that each experiment had to be chosen carefully, and many potentially useful follow-up ideas could not be tested within the available time and hardware constraints.

The dataset scope was also limited by the realities of the project. While the benchmark itself is challenging, training on larger subsets or running more extensive preprocessing pipelines increases runtime substantially. This makes it difficult to iterate quickly, especially when each major change may require feature regeneration and retraining. In a university project context, the tradeoff between dataset scale and experimental turnaround time is especially important. A larger dataset or more repeated evaluations would likely produce a more complete picture of model behavior, but the cost of running those experiments was high relative to the project timeline.

These limitations suggest several directions for future work. First, future experiments should be run on stronger hardware so that more systematic comparisons can be made across model sizes, learning rates, loss weights, and sequence lengths. Second, additional compute would make ablation studies much more practical. Because multiple fixes were introduced before the final Transformer run, it would be valuable to isolate the effects of improved normalization, missing-hand interpolation, temporal deltas, weighted sampling, and the hybrid classification-plus-CTC objective. This would provide further insight as to the relative value of each new feature, allowing for more precise isolation of further paths of study. Third, more computing resources and time would allow evaluation across multiple random seeds, which would make the reported gains more statistically reliable.

Future work could also expand the dataset scope in a more controlled way. For example, one could train and compare models across different vocabulary sizes or larger subsets while keeping the rest of the pipeline fixed. This would help determine how well the Transformer scales as the task becomes more difficult. Even with these limitations, the current experiments still support the central finding: within the realistic constraints of the project, the Transformer was a clear improvement over the BiLSTM baseline.

4. Conclusion

This project developed a landmark-based sign language captioning pipeline and evaluated how different temporal encoders affect sign prediction performance. The system converts input videos into normalized pose and hand landmark sequences, adds temporal motion features, and uses a temporal model to predict sign labels. We compared a BiLSTM baseline against a Transformer encoder while keeping the preprocessing pipeline and evaluation setup fixed.

The final results showed that the Transformer outperformed the BiLSTM across all reported metrics. It achieved lower test loss, higher exact-match accuracy, lower token error rate, and higher top-5 accuracy. This suggests that self-attention is better suited for capturing the temporal structure of signing motion than recurrent hidden-state propagation

in this setting. In particular, the Transformer can compare distant frames directly, which is useful because sign meaning often depends on motion patterns spread across the full clip.

However, the system is still not ready for real-world deployment. The final exact-match accuracy remains limited, and performance would need to improve significantly before the model could be used reliably in live accessibility settings. The project's current form can only caption pre-recorded videos, limiting its applicability as a real-time overlay, which would greatly improve efficiency in many real-world settings, such as online classrooms, hospital service desks, and gesture-based software interfaces. Future work should focus on larger datasets, stronger compute resources, more systematic hyperparameter tuning, ablation studies, and improved streaming evaluation. Despite these limitations, the project shows that improving both the preprocessing pipeline and temporal encoder can substantially improve sign language recognition performance.

References

[1] Necati Cihan Camgoz, Simon Hadfield, Oscar Koller, Hermann Ney, and Richard Bowden. Neural sign language translation. In *CVPR*, pages 7784–7793, 2018. 1, 2

[2] Necati Cihan Camgoz, Oscar Koller, Simon Hadfield, and Richard Bowden. Sign language transformers: Joint end-to-end sign language recognition and translation. In *CVPR*, pages 10023–10033, 2020. 2

[3] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, pages 6299–6308, 2017. 2

[4] Alex Graves, Santiago Fernandez, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, 2006. 2

[5] Dongxu Li, Cristian Rodriguez Opazo, Xin Yu, and Hongdong Li. Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1450–1459, 2020. 1